

Python: Estructuras, Funciones y POO

Unidad 4.2 — Una guía completa para dominar las estructuras de datos, la programación funcional y la orientación a objetos en Python. Ideal para desarrolladores con experiencia en Java que desean profundizar en el ecosistema Python.

UNIDAD 4.2 NIVEL INTERMEDIO



Listas y Tuplas

Las **listas** y las **tuplas** son las colecciones secuenciales más fundamentales de Python. Ambas almacenan elementos ordenados, pero difieren en mutabilidad: las listas son modificables, las tuplas son inmutables.

Listas: colecciones dinámicas

Características clave

- Ordenadas e indexadas desde 0
- Mutables: se pueden añadir, modificar y eliminar elementos
- Permiten elementos duplicados y tipos mixtos
- Operaciones: `append()`, `extend()`, `pop()`, `sort()`

Ejemplo práctico

```
frutas = ["manzana", "pera", "uva"]
frutas.append("mango")
frutas[1] = "melocotón"
frutas.sort()
print(frutas)
# ['mango', 'manzana', 'melocotón', 'uva']

# Slicing
primeras = frutas[:2]
# ['mango', 'manzana']
```

Diccionarios y Conjuntos

Los **diccionarios** almacenan pares clave-valor y permiten búsquedas en tiempo O(1). Los **conjuntos** son colecciones no ordenadas de elementos únicos, ideales para eliminar duplicados y realizar operaciones de teoría de conjuntos.



Diccionarios: mapas clave–valor

Operaciones esenciales

```
alumno = {
    "nombre": "Ana",
    "nota": 9.5,
    "módulos": ["Python", "SQL"]
}

# Acceso seguro
print(alumno.get("edad", "N/D"))

# Iterar
for clave, valor in alumno.items():
    print(f"{clave}: {valor}")

# Dict comprehension
cuadrados = {n: n**2 for n in range(5)}
```

Métodos útiles

- `keys()` — vista de claves
- `values()` — vista de valores
- `items()` — pares (clave, valor)
- `get(k, default)` — acceso seguro
- `update(otro_dict)` — fusionar
- `pop(k)` — eliminar y obtener

Desde Python 3.7 los diccionarios mantienen el orden de inserción.

Funciones Avanzadas

Python ofrece mecanismos muy flexibles para definir funciones: parámetros por defecto, argumentos posicionales variables (`*args`), argumentos de palabra clave variables (`**kwargs`) y funciones anónimas con `lambda`.



Parámetros y funciones lambda

Tipos de parámetros

```
def conectar(host, puerto=5432,
             *args, **kwargs):
    print(host, puerto)
    print(args)    # tupla extra
    print(kwargs)  # dict extra

conectar("localhost", usuario="ana",
         ssl=True)
```

- **Posicionales**: obligatorios, en orden
- **Por defecto**: valor predefinido
- ***args**: número variable de posicionales
- ****kwargs**: número variable de nombrados

Funciones lambda

Las **lambdas** son funciones anónimas de una sola expresión. Útiles como argumentos de funciones de orden superior.

```
doble = lambda x: x * 2
suma = lambda a, b: a + b

# Uso con sorted y map
alumnos = [("Ana", 9), ("Bea", 7)]
alumnos.sort(key=lambda a: a[1],
 reverse=True)

pares = list(map(lambda x: x*2,
 range(5)))
```

Ficheros: CSV y JSON

Trabajar con ficheros externos es imprescindible en cualquier aplicación real. Python proporciona módulos estándar para los dos formatos de intercambio de datos más habituales.

CSV con el módulo `csv`

```
import csv

# Lectura
with open("datos.csv", newline="",
          encoding="utf-8") as f:
    lector = csv.DictReader(f)
    for fila in lector:
        print(fila["nombre"])

# Escritura
with open("salida.csv", "w",
          newline="") as f:
    campos = ["nombre", "nota"]
    escritor = csv.DictWriter(f,
                fieldnames=campos)
    escritor.writeheader()
    escritor.writerow(
        {"nombre": "Ana", "nota": 9})
```

JSON con el módulo `json`

```
import json

datos = {"nombre": "Ana", "nota": 9.5}

# Serializar a fichero
with open("datos.json", "w") as f:
 json.dump(datos, f, indent=2)

# Deserializar desde fichero
with open("datos.json") as f:
 cargado = json.load(f)

# Serializar a cadena
cadena = json.dumps(datos)
objeto = json.loads(cadena)
```

Manejo de Excepciones

Las excepciones permiten manejar errores en tiempo de ejecución de forma controlada, evitando que el programa termine abruptamente. Python usa el bloque `try / except / else / finally` para gestionar errores de manera estructurada.



Clases y Objetos en Python

Python es un lenguaje orientado a objetos de primera clase. Sus clases son **dinámicas**, más flexibles que en Java, y soportan herencia múltiple, métodos especiales (*dunder methods*) y propiedades con decoradores.



Herencia y polimorfismo

Herencia en Python

```
class Persona:
    def __init__(self, nombre):
        self.nombre = nombre

    def saludar(self):
        return f"Hola, soy {self.nombre}"

class Docente(Persona):
    def __init__(self, nombre, dpto):
        super().__init__(nombre)
        self.dpto = dpto

    def saludar(self):  # polimorfismo
        base = super().saludar()
        return f"{base}, del dpto. {self.dpto}"

class Investigador(Persona):
    pass  # hereda todo sin cambios

# Herencia múltiple
class ProfesorInvestigador(Docente,
                           Investigador):
    pass
```

Conceptos destacados

Buenas Prácticas: PEP 8 y Docstrings

El **PEP 8** es la guía de estilo oficial de Python. Seguirla hace que el código sea más legible y mantenible. Los **docstrings** documentan módulos, clases y funciones de forma estándar, habilitando herramientas como `help()` y Sphinx.

Nomenclatura

- `snake_case` para variables y funciones
- `PascalCase` para clases
- `UPPER_CASE` para constantes
- `_privado` por convención

Formato

- Máximo 79 caracteres por línea
- 4 espacios de indentación (no tabs)
- 2 líneas en blanco entre funciones de nivel superior
- Espacios alrededor de operadores

Docstrings

```
def calcular_media(notas):
    """
    Calcula la media aritmética.

    Args:
    notas (list[float]): Lista
    de calificaciones.

    Returns:
    float: Media de las notas.

    Raises:
    ValueError: Si la lista está
    vacía.
    """
```

S04 · EN EL REPOSITORIO DEL CURSO

Ahora, a practicar

Lo que viene a continuación está en tu repositorio; no hace falta buscar ejercicios fuera.

RETO

Reto del sprint — 130 min

Preparar la carpeta y ver que las pruebas fallan → rellenar las funciones una a una → exportar y ejecutar el…

sprints/S04/guia/Reto_Sprint_04.md